# Backends for mobile

Serverless, a VPS, and Go

**Dinu-Ștefan Rusu**
FILS, Universitatea Politehnica București · Spring 2027

TODAY

# What you should leave with

### Split client and server

What must run on a machine you control, and why the client is untrusted input.

### Choose serverless or a VPS

Cold starts, scaling and cost shape, and the case where each one wins.

### Design an API for mobile

Pagination, versioning, conditional requests, compression and rate limits.

### Ship a Go backend

A minimal Docker image, a systemd unit, environment variables, health checks and logs.

PART 1 · SERVER AND CLIENT

# Where does the code run?

The same feature costs different things on either side of the wire

# Server-side and client-side execution

Client-side code runs on the phone: rendering, local state, caching, and the optimistic update that makes a form feel instant.

Server-side code runs on a machine you control: secrets, durable storage, authorization, billing, and anything that must be identical for every user.

One request and one response cross the network between them. On mobile that round trip costs roughly 40–200 ms on a good connection, seconds on a bad one, and nothing at all inside a tunnel.

REQUEST LIFECYCLE

1. Client builds the request and holds the token.
2. Network: a handshake, then the round trip.
3. Server: authenticate, authorize, execute.
4. Response: a status code and a body.

**Put work on the client for latency.** Put it on the server when the answer has to be correct for every user.

# The client is untrusted input

Everything that arrives from a client is user input, including fields your own app filled in.

An Android package (APK) can be unpacked and release Dart code inspected. There is no secret you can ship inside a client.

Client-side validation is a courtesy: an instant warning, a disabled button, no wasted round trip.

The server repeats every check it cares about; it is the only copy nobody else can edit.

**Server-side only**

API keys, and anything signed with one. Who may read or write which record. Prices, totals, discounts, balances. Rate limits and quotas.

**Validate on the client for speed, on the server for correctness.** Any rule enforced only in the app can be bypassed.

PART 2 · WHERE THE SERVER RUNS

# Serverless vs a VPS

Two cost shapes, two failure modes, and two different sets of operational work

# Serverless: functions as a service

Functions as a Service (FaaS) means you deploy a function, not a machine. The platform provisions, scales, patches, and terminates Transport Layer Security (TLS) connections for you.

It creates instances on demand and destroys them when traffic stops, scaling all the way down to zero.

Billing is per invocation plus gigabyte-seconds of memory, so no traffic means no bill.

AWS Lambda, Google Cloud Run functions, Azure Functions, and Cloudflare Workers all follow this model.

THE SHAPE OF IT

1. Your handler: one function, one job.
2. The platform: provision, scale, patch, terminate TLS.
3. 0 to n instances, killed when idle.

**You stop operating servers.** In exchange you work within the platform's limits.

SERVERLESS

# Cold starts: the latency of the first request

| CASE | STARTUP | WHY |
|---|---|---|
| Warm instance | 0 ms | the container is already running |
| Go or Rust binary | 10–50 ms | nothing to boot: one static executable |
| JVM, .NET, large Node graphs | 0.3–2 s | runtime start, plus every import |

A cold start is the platform building a new instance because no warm one was free: pull the image, start the runtime, run top-level code, then finally the handler.
It lands on the first user after a period of no traffic, and again on every traffic spike.

Minimum instances, or provisioned concurrency, mean paying for idle capacity, the exact cost serverless was meant to remove. A smaller package and a fast-starting runtime cost nothing extra, which is one reason for the Go section later in this lecture.

# VPS: renting the whole machine

A Virtual Private Server (VPS) is a virtual machine you rent from a provider such as Hetzner Cloud, DigitalOcean, or OVH: root access, one IP address, and nothing else.

Everything above the hypervisor is yours: operating system updates, firewall rules, TLS certificates, process supervision, backups, monitoring.

Billing is hourly or monthly for the machine, not for the requests it serves. An idle VPS costs what a busy one costs.

THE SHAPE OF IT

1. Your binary: systemd or Docker restarts it.
2. The OS you patch: firewall, TLS certificates.
3. The VM you rent: CPU, RAM, disk, a public IP.

**Nobody patches it for you.** In exchange, no platform limits what you can run.

SERVERLESS VS VPS

# The same backend, two cost shapes

| ASPECT | SERVERLESS (FAAS) | VPS |
|---|---|---|
| Scaling | automatic, zero to peak and back | you resize the box yourself |
| Cost when idle | nothing | the full monthly price |
| Cost under load | per invocation and memory time | flat, until you outgrow the machine |
| First request | a cold start, unless paid to avoid it | already warm |
| You operate | your handler | OS, runtime, TLS, backups, monitoring |
| Long-running work | capped by the platform timeout | runs as long as you let it |
| Wins when | spiky, small, or unknown traffic | steady traffic, or state on disk |

**Pick by cost shape and operational load.** Spiky or unknown favors serverless. Steady and predictable favors one small VPS.

SERVERLESS

# Writing code that survives a FaaS runtime

### Ship a small package

Every megabyte and import adds cold-start time. Trim dependencies first.

### Initialize outside the handler

Database pools and HTTP clients belong at module scope, reused by warm invocations.

### Keep the handler stateless

Local disk and memory vanish between calls. State belongs in a database or a cache.

### Make it idempotent

The platform retries on its own. A duplicate must not charge a card twice.

### Respect the timeout

Work that can exceed it belongs on a queue with a job record, not a longer function.

### Know your concurrency

One provider serves one request per instance; another serves many at once.

# Running a box you will not regret

### Rebuild it from a script

A Dockerfile plus one provisioning script. The machine should be reproducible, never hand-tuned.

### Run it as a service

systemd or Docker with a restart policy. A crash at 3 a.m. must recover without you.

### Lock down access

Key-only SSH, no root login, a firewall open only on ports 80 and 443, automatic security updates.

### Terminate TLS properly

Caddy or nginx in front, with automatic certificates. Never serve an API over plain HTTP.

### Test the restore

Snapshots plus an off-box copy of the database. A backup you never restored is unverified.

### Make it page you

An uptime check, log shipping, a disk-full alert. A bare VPS does not tell you it went down.

PART 3 · GO BACKENDS

# A backend that fits in one binary

One static binary, milliseconds to start, deployable anywhere

# Why Go for a small mobile backend

**One static binary.**
One executable: no runtime, no dependency folder.
**It starts in milliseconds.**
Cold starts on a scale-to-zero platform stay short.
**It cross-compiles from your laptop.**
One variable retargets it: a server or a Pi.
**The standard library is already the server.**
net/http, encoding/json, goroutines (Lecture 2).

```
CGO_ENABLED=0 GOOS=linux \
  GOARCH=arm64 \
  go build -o server
scp server \
  root@vps:/opt/api/
ssh root@vps \
  systemctl restart api
```

**A small binary that starts fast.** Those are the properties a mobile client's backend needs.

# A handler, and the Dart call that hits it

### Go: standard library only

```go
type Reading struct {
    DeviceID string `json:"device_id"`
}
func handle(w http.ResponseWriter,
    r *http.Request) {
    var in Reading
    json.NewDecoder(r.Body).Decode(&in)
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(in)
}
```

### Dart: the client side of the call

```dart
final dio = Dio(BaseOptions(
  baseUrl:
    'https://api.example.com',
));
final res = await dio.post(
  '/readings',
  data: {'device_id': id},
);
```

`Reading.fromJson(res.data)` decodes the reply. The JSON tags are the contract between Go and Dart.

# Thundering herds and hedged requests

When a service fails briefly, ten thousand clients fail at once and every one retries after exactly one second. The retry wave is larger than the original load: a brief outage becomes a long one. That is a thundering herd.

Full jitter spreads retries across the whole interval, so a recovering server sees load ramp up instead of arriving all at once.

Hedging is the opposite problem: a small fraction of requests are simply slow. Send a second copy after a short delay, take whichever answers first, and cancel the other.

| ATTEMPT | RESULT |
|---|---|
| sent at 0 ms | silent at 250 ms, canceled |
| sent at 250 ms | answers at 310 ms, used |

Cap it: hedge only past the 95th percentile, and only for safe or idempotent calls.

**Retries fix failures. Hedges fix slowness.** Both multiply load, so both need a cap.

PART 4 · API DESIGN FOR A MOBILE CLIENT

# Design decisions clients actually feel

Pagination, versioning, conditional requests, compression, and rate limits

# Pagination with cursors, not page numbers

An offset-and-limit page breaks when rows are inserted or deleted between requests: page two can skip or repeat rows.

A cursor is an opaque token that encodes the last item's sort key. The client sends it back to get the next page; nothing else has to stay stable.

Cursors compose with real-time inserts, work with any sort order, and never need a total row count the database would have to compute.

```
// cursor, limit: query params
type Page struct {
    Items []Reading
    `json:"items"`
    Next  string
    `json:"next,omitempty"`
}
```

**A cursor is a fact about the last row returned.** It is not a page count the server has to keep consistent across writes.

# Versioning: give clients time to update

A mobile client cannot be forced to update. Old app versions can stay installed for years, so the server must keep serving them.

Put the version in the URL path, like `/v1/readings`, or in an Accept header. A path is simpler to log, cache, and route.

Add fields freely: a client that ignores unknown JSON fields never breaks. Never remove or rename a field, and never change what a field means, without a new version.

Deprecate loudly: a `Sunset` header and a fixed shutdown date, announced early and enforced only after clients had time to move.

**Additive changes are free.** Anything else needs a new version number.

# ETags and conditional requests

An entity tag (ETag) is a short identifier for one exact representation of a resource: a hash or a version number.
The server returns `ETag`. The client sends it back as `If-None-Match`; unchanged means 304 Not Modified, no body.
`If-Match` on a write rejects it with 412 Precondition Failed if the resource changed since it was last read.

| HEADER | DIRECTION | MEANING |
|---|---|---|
| ETag | response | fingerprint of this representation |
| If-None-Match | request | send a new copy only if this changed |
| If-Match | request | write only if it still matches this version |

**Conditional requests turn a download into a status code.** And a blind write into a safe one.

# Compression: fewer bytes on a metered radio

A phone often pays for every byte in time, battery, and sometimes money. Compression trades a little CPU time for fewer bytes on the radio.

The client sends `Accept-Encoding: gzip`. The server compresses the body and answers with `Content-Encoding: gzip`. Most HTTP client libraries decompress transparently.

JSON compresses well: repeated keys and whitespace shrink hard, often 70–90 percent smaller. A response already under about 1 KB rarely benefits enough to bother.

Do not compress what is already compressed: images, video, and most binary formats gain nothing and waste CPU cycles.

**Compress text responses by default.** Skip it for anything already binary.

# Rate limits: protecting the server from its clients

A rate limit caps how many requests one client, one API key, or one IP address may make in a window, so one runaway client or a retry storm cannot take the service down for everyone else.

The token bucket is the common shape: a bucket refills at a fixed rate, and each request costs one token; a burst is allowed until the bucket is empty.

The server answers 429 Too Many Requests with a `Retry-After` header, so a well-behaved client waits exactly that long instead of guessing.

Return the budget in headers on every response, such as `X-RateLimit-Remaining`, so a client can back off before it gets rejected.

**A limit the client can see in headers is one clients cooperate with.** Not one they fight.

PART 5 · DEPLOYING AND OPERATING

# Shipping the binary and keeping it healthy

A Docker image, Cloud Run, systemd, secrets, health checks, and logs

# A minimal Dockerfile for a Go binary

```
FROM golang:1 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /server ./cmd/server

FROM gcr.io/distroless/static
COPY --from=build /server /server
EXPOSE 8080
ENTRYPOINT ["/server"]
```

Multi-stage build: the first stage compiles with the full Go toolchain; the second stage copies only the binary into a distroless image with no shell and no package manager, which shrinks the image and the attack surface.

# The same image, deployed to Cloud Run

`gcloud run deploy` builds and deploys the container behind a managed load balancer. Nobody patches the machine.

Concurrency, instances, memory, and CPU are flags, not hand-provisioned infrastructure.

A new revision gets traffic gradually, so a bad deploy rolls back in seconds.

```
gcloud run deploy api \
  --source . \
  --region europe-west1 \
  --set-env-vars ENV=production \
  --min-instances=0 --max-instances=10
```

**Cloud Run is the serverless model from Part 2.** With your own container, not a vendor runtime.

# A systemd unit on a VPS

```
[Unit]
Description=api backend
After=network.target

[Service]
ExecStart=/usr/local/bin/server
Restart=always
RestartSec=2
User=api
EnvironmentFile=/etc/api/api.env

[Install]
WantedBy=multi-user.target
```

**One file, one service.**

`systemctl enable --now api` starts it and keeps it running across reboots.

`Restart=always` with a short `RestartSec` means a crash recovers in about two seconds, with nobody paged at 3 a.m.

# Environment variables and secrets

Configuration that changes between environments belongs in environment variables, read once at startup, never hardcoded.

Secrets are configuration too, but never in the code or the git repository. A leaked key is compromised the moment it is pushed, even after deletion.

On a VPS: a root-only environment file loaded by systemd. On Cloud Run: a secret manager, never baked into the image.

```
dsn :=
    os.Getenv("DATABASE_URL")
if dsn == "" {
    log.Fatal("DATABASE_URL
    is required")
}
```

**A required variable that is missing should crash at startup.** Not on the first request, hours later.

# Health checks: liveness and readiness

Liveness answers one question: is the process still running and not deadlocked. A failing liveness check gets the process restarted, so keep it cheap and dependency-free. Readiness answers a different question: is this instance ready for traffic. Failing it removes the instance from the load balancer without a restart.
Cloud Run, most orchestrators, and load balancers expect a plain HTTP endpoint that returns 200 when healthy, checked every few seconds.

```
http.HandleFunc("/healthz", func(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
})
```

**Liveness restarts the process. Readiness redirects traffic.** Confusing the two either restarts a healthy process or sends traffic to a broken one.

# Structured logs: fields, not sentences

A log line meant for a person cannot be filtered, aggregated, or alerted on reliably. A structured log line is a set of key-value fields a machine can query.

Go's log/slog, in the standard library, writes JSON by default: a level, a message, and any fields attached, one line per event.

Always include a request identifier a client sends or the server generates, so one failing mobile session can be traced through every log line it touched.

```
h := slog.NewJSONHandler(
  os.Stdout, nil,
)
logger := slog.New(h)
logger.Info("reading saved",
  "device_id", id,
  "duration_ms", ms)
```

**If you cannot turn it into a chart, it is not a structured log yet.** A sentence in a log file is for a human reading it live, once.

# What the project backend needs

Lab 5's sync server is this lecture in miniature: a small Go program, an in-memory list with a version per row, one endpoint that accepts offline writes and returns what changed since a cursor.
Lab 6 adds gRPC (Lecture 11) to the same binary: one unary call and one server-streaming call, both served from the process that already answers HTTP.
Keep it on one small VPS or one Cloud Run service for the whole semester. The project needs a reliable box, not a distributed system.
Everything in this lecture applies at project scale: environment variables for broker and database URLs, a health check the grading script can poll, and structured logs for the night before the defense.

**The project backend is small on purpose.** The skill being graded is running it correctly, not scaling it.

RECAP

# Three things to remember

1. The client is untrusted input. Every rule that matters is checked again on the server.
2. Serverless bills per request and cold-starts; a VPS bills per month and never does. Pick by cost shape, not by fashion.
3. A cursor, an ETag, and a rate-limit header are what make an API pleasant to build a mobile client against. Small, additive changes keep old app versions working.

FURTHER READING

go.dev/doc/tutorial/web-service-gin · pkg.go.dev/net/http · pkg.go.dev/log/slog · cloud.google.com/run/docs

# Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel